2차 프로젝트 발표
IPTV 채널 추천 팀

박종현, 권정인, 신새봄, 이중찬, 정연진

목차

1 프로젝트 방향성
2 개발 환경
3 팀원 역할 분배
4 기술 플로우
5 시스템 아키텍쳐
6 현재 진행 상황

프로젝트 방향성

기존

사용자의 시청 이력 패턴을 분석해,
유사한 장르의 채널 추천 혹은
유사하지 않은 장르의 채널 추천 진행

채널 추천 방식 수정 결정

1 사용자 데이터 수집 한계

사용자 데이터 확보하지 못할 가능성 有

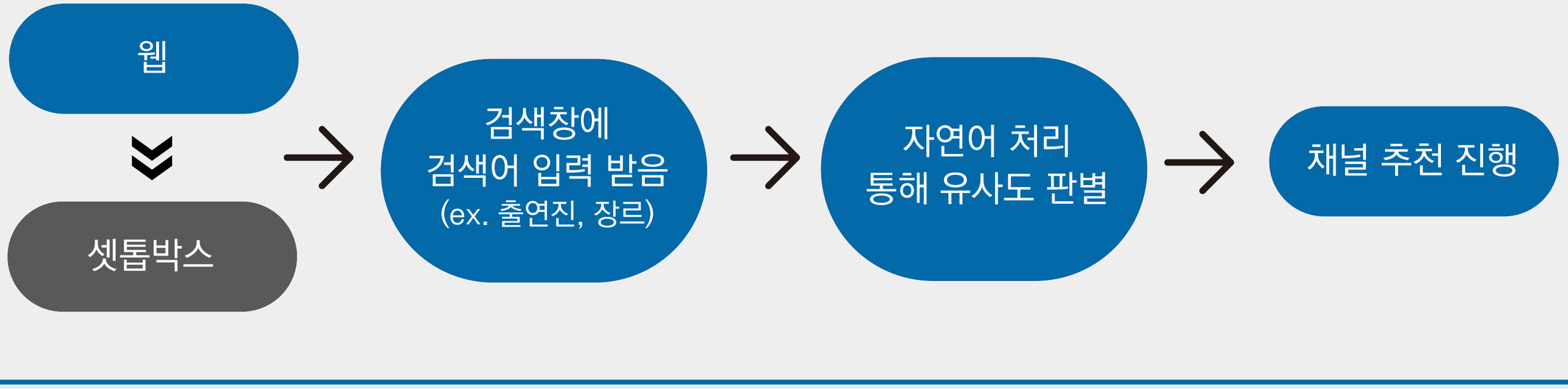
2 프로그램 정보 수집 어려움

텍스트뿐만 아니라 이미지로 구성되어 있어 텍스트 추출 어려움

설명 부족한 일부 프로그램의 경우 충분한 정보 얻기 어려움

프로젝트 방향성

목표 : 검색 기능을 통한 사용자 맞춤형 채널 추천



개발 환경

Backend

- Django
- Java SpringBoot

Frontend

- HTML
- CSS
- JS(React)

Infra(AWS)

- S3
- RDS
- EC2
- Lambda
- API Gateway
- EventBridge

Tools

- Jupyter Notebook
- PyCharm
- Google Colab
- Android Studio
- Visual Studio Code

Collaborations

- Git
- Discord
- Notion
- Draw.io

Database

- MySQL
- Oracle

팀원 역할 분배

박종현

인프라 아키텍처 및 소프트웨어 개발

권정인

기획, AI 테스팅, 프로젝트 자료 구성

신새봄

AI 모델 구축 및 학습

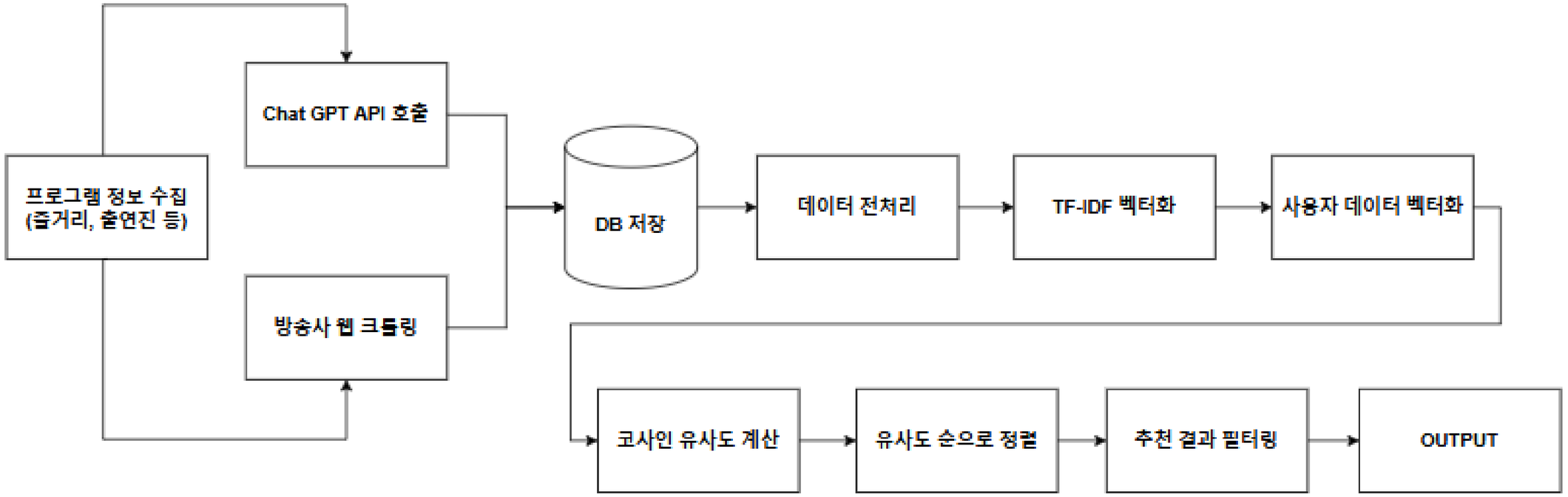
정연진

AI 모델 구축 및 학습

이중찬

인프라 아키텍처 설계 및 구축

기술 플로우

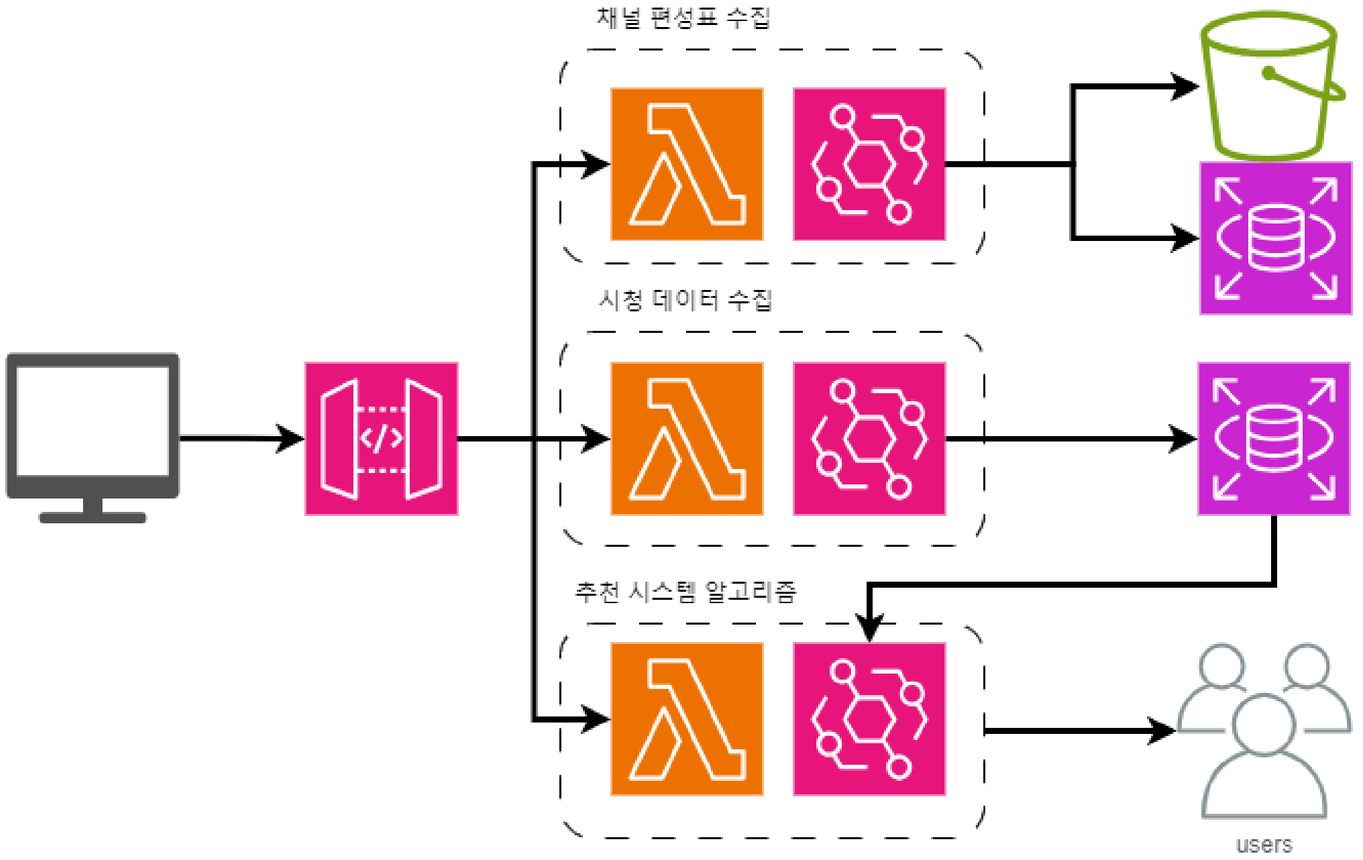


* TF (Term Frequency, 단어 빈도)
특정 문서 내에서 특정 단어가 얼마나 자주 나타나는지를 측정

* IDF (Inverse Document Frequency, 역문서 빈도)
특정 단어가 얼마나 다른 문서에서 자주 나타나는지를 측정

* 코사인 유사도 : 사용자 입력값이랑 우리가 추출한 정보와의 유사도를 판별하는 하나의 기법

시스템 아키텍처



현재 진행 상황_웹 크롤링

라이브러리 불러오기

```
from bs4 import BeautifulSoup
from seleniumwire import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time
import pandas as pd
from selenium.webdriver.common.by import By
```

LG U+ 채널 사이트에서 크롤링 진행

```
# 크롤링할 웹페이지 URL 설정
url = "https://www.lguplus.com/iptv/channel-guide/schedule"
chrome_options = webdriver.ChromeOptions()
# Chrome 드라이버 실행
browser = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=chrome_options)
time.sleep(1)
browser.get(url)  # 웹페이지 로드
```

현재 진행 상황_웹 크롤링

반복문 사용해 전체 352개의 채널에 편성된 티비 프로그램 정보 수집

```
for i in range(1, 352):
    # 다음 페이지로 이동 버튼 클릭
    browser.find_element(By.XPATH,
                         value: "/html/body/div[1]/div/div/div[4]/div[1]/div/div[2]/div/div/div/div[2]/div/div/div[2]/div/div[1]/div[3]/a").click()
    time.sleep(1)
    clickchannel = browser.find_element(By.XPATH,
                                        "/html/body/div[1]/div/div/div[4]/div[1]/div/div[2]/div/div/div/div[2]/div/div/div[2]/div/div[1]/div[2]/ul/li[" + str(
                                            i) + "]/a")
    # 화면 확대 축소 설정
    browser.execute_script("document.body.style.zoom='10%'")
    # 채널 클릭하여 해당 채널 데이터 수집
    clickchannel.click()
    time.sleep(1)
```

현재 진행 상황_웹 크롤링

반복문 사용해 전체 352개의 채널에 편성된 티비 프로그램 정보 수집

```
# 페이지 소스 가져오기 및 파싱
html = browser.page_source
time.sleep(1)
soup_temp = BeautifulSoup(html, features: 'html.parser')
txt4page = len(soup_temp.findAll("td"))  # 'td' 태그 개수 확인 (디버깅용 출력)
print(txt4page)

# 채널 이름 추출 및 저장
channelname = soup_temp.find( name: 'h3', attrs: 'h3')
srcURL2.append(["", channelname.get_text().replace( _old: '<h3 class="h3">', _new: "").replace( _old: "</h3>", _new: ""), ""])

# 채널의 프로그램 데이터를 'td' 태그에서 추출
html = browser.page_source
soup = BeautifulSoup(html, features: 'html.parser')
txt1 = soup.findAll("td")

cellcount = 0  # 2개의 데이터마다 리스트에 저장하기 위한 카운터 초기화
```

현재 진행 상황_웹 크롤링

TD태그(HTML에서 표를 구성하는 태그)에서 프로그램명, 시간, 장르 수집

```
# 추출한 프로그램 데이터를 텍스트로 변환 후 리스트에 저장
for line in txt1:
    linetxt = line.get_text()
    print(fileNum, " : ", linetxt)
    if len(linetxt) > 40:
        sindex = linetxt.find("<div>")
        content = linetxt[sindex + 5:40].split("<small")[0]
    else:
        content = linetxt.replace( _old: '<td class="" role="cell">', _new: "").replace( _old: '</td>', _new: "")
        cellcount += 1
    srcURL1.append(content)
    fileNum += 1
    if cellcount == 2:  # 2개 데이터를 묶어서 저장 후 초기화
        srcURL2.append(srcURL1)
        cellcount = 0
        srcURL1 = []
fileNum = 0
```

```
# 수집한 데이터를 데이터프레임으로 변환 및 Excel 파일로 저장
df = pd.DataFrame(srcURL2, columns=["Time", 'Title', "Genre"])
df.head()
df.to_excel( excel_writer: r'Title Scrap1.xlsx', index=False)
```

시연

현재 진행 상황_웹 크롤링 결과값

전체 352개의 채널에서 채널명, 방영 시간, 방영 프로그램을 성공적으로 가져오는 것을 확인

	KBS드라마[31]	
00:10:00	1박 2일 시즌4 (247회)	연예/오락
01:30:00	개그 콘서트 (1094회)<재>	연예/오락
02:50:00	신상출시 편스토랑 (246회)<재>	연예/오락
03:50:00	걸어서 세계속으로 [알로아 하와이1 - 오아후,마	뉴스/정보
04:50:00	결혼하자 맹꽁아! (13회)<재>	드라마
05:20:00	결혼하자 맹꽁아! (14회)<재>	드라마
06:00:00	결혼하자 맹꽁아! (15회)<재>	드라마
06:25:00	결혼하자 맹꽁아! (16회)	드라마
07:10:00	스캔들 (75회)<재>	드라마
07:40:00	스캔들 (76회)<재>	드라마
08:30:00	스캔들 (77회)<재>	드라마
09:00:00	스캔들 (78회)<재>	드라마
09:50:00	박원숙의 같이 삽시다 3 (184회)<재>	연예/오락
11:10:00	신상출시 편스토랑 (246회)<재>	연예/오락
12:30:00	스캔들 (77회)<재>	드라마
13:05:00	스캔들 (78회)<재>	드라마
13:55:00	다리미 패밀리 (9회)<재>	드라마
15:15:00	다리미 패밀리 (10회)<재>	드라마
16:35:00	이십세기 힛-트쏭 (237회)<재>	공연/음악
17:50:00	살림하는 남자들 2 삼천포 서진이네 (12회)<재	연예/오락
19:15:00	차트를 달리는 남자 (410회)<재>	연예/오락
20:25:00	무엇이든 물어보살 (289회)	연예/오락
21:45:00	1박 2일 시즌4 (247회)<재>	연예/오락
23:00:00	살림하는 남자들 2 (378회)<재>	연예/오락

	채널A[18]	
00:30:00	28청춘 (17회)<재>	뉴스/정보
01:30:00	채널A 스페셜 그랜드디자인-집 개조 프로젝트 (1	뉴스/정보
02:50:00	채널A 스페셜 세계 로컬 여행기 (5회)	뉴스/정보
03:50:00	채널A 화재 특보<재>	다큐
04:00:00	채널A 지진 특보<재>	다큐
04:10:00	채널A 스페셜 브라보K 사이언티스트 (173회)	뉴스/정보
04:40:00	채널A 스페셜 정원 가꾸기 in 호주 시즌32 (	뉴스/정보
06:00:00	100세 프로젝트 (158회)<재>	뉴스/정보
07:00:00	새콤달콤 캐치! 티니핑 (4회)	만화
07:30:00	행복한 아침	뉴스/정보
08:50:00	김진의 돌직구 쇼	뉴스/정보
10:30:00	강철부대W (4회)<재>	연예/오락
12:00:00	뉴스A LIVE	뉴스/정보
13:20:00	100세 프로젝트 (151회)<재>	뉴스/정보
14:20:00	두둥탁! 이야기가 얼쑤 (10회)<재>	공연/음악
14:50:00	요즘 남자 라이프 신랑수업 스페셜 (135회)<재	연예/오락
15:50:00	이야기 더	뉴스/정보
17:10:00	뉴스 TOP10	뉴스/정보
19:00:00	뉴스A	뉴스/정보
20:10:00	몸신의 탄생 (6회)	뉴스/정보
21:20:00	절친 토큐멘터리 4인용식탁 스페셜 (5회)	뉴스/정보
22:00:00	강철부대W (5회)	연예/오락
23:50:00	요즘 남자 라이프 신랑수업 (136회)<재>	연예/오락

현재 진행 상황

프로그램명을 입력하면 출연진 정보를 알려주는 프로그램을 OpenAI를 활용해 제작

1. 라이브러리 임포트 및 API 키 설정

```
import openai
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import time

# ChatGPT API 키 설정
openai.api_key = "sk-proj-4_gQSqjIpdYpSDp1U6JSemyx3k0prUTBjpz6wAAHuL703DldSzgqaOckyIXDRtzdMYEQmFm4tzT3BlbkFJW799bB93Ousj1KxlCQI84q7f6cECe4hMyX2hYo_2mAn1w
```

2. 프로그램 기본 정보 설정 (웹크롤링 된 데이터 4개의 행)

```
# 웹 크롤링을 통해 얻은 프로그램 기본 정보
data = {
    'program_name': ['유 퀴즈 온 더 블럭', '지옥에서 온 판사', '최강야구', '한블리(한문철의 블랙박스 리뷰)'],
    'genre': ['예능', '드라마', '예능', '시사/교양'],
    'broadcast_time': ['20:45', '22:00', '22:30', '20:50']
}
df = pd.DataFrame(data)
```

현재 진행 상황

프로그램명을 입력하면 출연진 정보를 알려주는 프로그램을 OpenAI를 활용해 제작

3. 프로그램 설명 생성 함수 정의

```python
def get_cast_info(program_name):
    """
    특정 프로그램의 출연진 정보를 가져옵니다.
    """
    prompt = f"TV 프로그램 '{program_name}'의 주요 출연진을 한글로 제공해 주세요. 출연진은 '출연진: ' 형식으로만 답변해 주세요."
    response = openai.ChatCompletion.create(
        model="gpt-4-turbo",
        messages=[
            {"role": "system", "content": "You are a knowledgeable assistant for providing TV program information."},
            {"role": "user", "content": prompt}
        ],
        max_tokens=150,
        temperature=0.7
    )
    details = response['choices'][0]['message']['content'].strip()
    cast_info = details.split("출연진:")[1].strip() if "출연진:" in details else "출연진 정보 없음"
    return cast_info
```

현재 진행 상황

프로그램명을 입력하면 출연진 정보를 알려주는 프로그램을 OpenAI를 활용해 제작

```python
def get_program_description(program_name):
    """
    특정 프로그램의 설명을 가져옵니다.
    """
    prompt = f"TV 프로그램 '{program_name}'의 내용을 간단히 설명해 주세요."
    response = openai.ChatCompletion.create(
        model="gpt-4-turbo",
        messages=[
            {"role": "system", "content": "You are a knowledgeable assistant for providing TV program information."},
            {"role": "user", "content": prompt}
        ],
        max_tokens=300,
        temperature=0.7
    )
    description = response['choices'][0]['message']['content'].strip()
    return description
```

현재 진행 상황

프로그램명을 입력하면 출연진 정보를 알려주는 프로그램을 OpenAI를 활용해 제작

4. 각 프로그램에 대해 설명 및 출연진 정보 추가

```
# 프로그램 설명 및 출연진 생성 후 데이터프레임에 추가
df['cast'] = None  # 출연진 컬럼 추가
df['description'] = None  # 설명 컬럼 추가

for i in range(len(df)):
    program_name = df.loc[i, 'program_name']
    genre = df.loc[i, 'genre']
    broadcast_time = df.loc[i, 'broadcast_time']
    try:
        # 출연진 정보 요청
        cast = get_cast_info(program_name)
        time.sleep(1)  # 호출 간격 조정

        # 프로그램 설명 요청
        description = get_program_description(program_name)
        time.sleep(1)  # 호출 간격 조정

        # 데이터프레임에 결과 저장
        df.loc[i, 'cast'] = cast
        df.loc[i, 'description'] = description

        print(f"{program_name} 설명 및 출연진 생성 완료")
    except Exception as e:
        print(f"{program_name} 설명 생성 중 오류: {e}")
```

```
유 퀴즈 온 더 블럭 설명 및 출연진 생성 완료
지옥에서 온 판사 설명 및 출연진 생성 완료
최강야구 설명 및 출연진 생성 완료
한블리(한문철의 블랙박스 리뷰) 설명 및 출연진 생성 완료
```

유 퀴즈 온 더 블럭 설명 및 출연진 생성 완료
지옥에서 온 판사 설명 및 출연진 생성 완료
최강야구 설명 및 출연진 생성 완료
한블리(한문철의 블랙박스 리뷰) 설명 및 출연진 생성 완료

현재 진행 상황

프로그램명을 입력하면 출연진 정보를 알려주는 프로그램을 OpenAI를 활용해 제작

5. 사용자 검색어 기반 TF-IDF 벡터화 및 코사인 유사도 계산

```
# 사용자 입력 받기
user_input = input("추천 받고 싶은 프로그램에 대한 키워드를 입력하세요: ")

# 출연진과 설명 정보를 합쳐서 유사도 계산을 위한 텍스트로 사용
df['combined_info'] = df['cast'] + " " + df['description']

# 사용자 입력과 프로그램 정보 결합
combined_texts = df['combined_info'].tolist() + [user_input]

# TF-IDF 벡터화
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(combined_texts)

# 코사인 유사도 계산 (사용자 입력과 프로그램 정보 간의 유사도)
cosine_similarities = cosine_similarity(tfidf_matrix[-1:], tfidf_matrix[:-1]).flatten()

# 유사도가 가장 높은 프로그램 선택
most_similar_index = cosine_similarities.argmax()
most_similar_program = df.iloc[most_similar_index]

# 결과 출력
print("\n추천 프로그램:")
print(f"프로그램명: {most_similar_program['program_name']}")
print(f"장르: {most_similar_program['genre']}")
print(f"방송 시간: {most_similar_program['broadcast_time']}")
print(f"출연진: {most_similar_program['cast']}")
print(f"설명:")
print(most_similar_program['description'])
print(f"유사도 점수: {cosine_similarities[most_similar_index]:.2f}")
```

```
추천 받고 싶은 프로그램에 대한 키워드를 입력하세요:  유재석

추천 프로그램:
프로그램명: 유 퀴즈 온 더 블럭
장르: 예능
방송 시간: 20:45
출연진: 유재석, 조세호
설명:
'유 퀴즈 온 더 블럭'은 대한민국의 텔레비전 예능 프로그램으로, 유재석과 조세호가 진행을 맡고 있습니다. 이 프로그램은 두 MC가 대한민국의 다양한 지역을 방문하여 거리에서 만난 사람들과 대화를 나누고, 그들의 이야기를 듣는 형식으로 진행됩니다. 또한, 일반인 참가자들에게는 퀴즈를 제공하여 정답을 맞히면 상금을 주는 코너도 포함되어 있습니다. 이 프로그램은 다양한 사람들의 삶의 이야기와 지혜를 공유하며, 휴먼 다큐멘터리와 예능이 결합된 독특한 형식을 가지고 있습니다.
유사도 점수: 0.09
```


```
추천 받고 싶은 프로그램에 대한 키워드를 입력하세요:  유재석

추천 프로그램:
프로그램명: 유 퀴즈 온 더 블럭
장르: 예능
방송 시간: 20:45
출연진: 유재석, 조세호
설명:
'유 퀴즈 온 더 블럭'은 대한민국의 텔레비전 예능 프로그램으로, 유재석과 조세호가 진행을 맡고 있습니다. 이 프로그램은 두 MC가 대한민국의 다양한 지역을 방문하여 거리에서 만난 사람들과 대화를 나누고, 그들의 이야기를 듣는 형식으로 진행됩니다. 또한, 일반인 참가자들에게는 퀴즈를 제공하여 정답을 맞히면 상금을 주는 코너도 포함되어 있습니다. 이 프로그램은 다양한 사람들의 삶의 이야기와 지혜를 공유하며, 휴먼 다큐멘터리와 예능이 결합된 독특한 형식을 가지고 있습니다.
유사도 점수: 0.09
```

시연

Q&A

추후 계획

현재 상태와 발전 방향성

2차 프로젝트

- 람다 최대 사용 시간이 15분, 그러나 현재 크롤링 데이터 처리 위해선 최소 20분 소요
  따라서 서비스 안정화를 완벽하게 확보하지 못한 상황

3차 프로젝트

- 람다 기능 안정화 확보
  ▫ 채널 분할: 많은 채널들을 일정 개수로 분리하여, 람다 함수를 더 추가하여 크롤링 예정

- **사용자 데이터 확보**가 가능하다면, 2가지 기준도 추천 시 함께 고려
  ▫ 시청 시간: 해당 채널을 얼마나 오래 시청했는지
  ▫ 선호 장르: 시청한 프로그램 중 어떤 장르가 가장 많은지